

C++课程设计项目报告

图书借阅管理系统

封面

项目	内容
项目名称	图书借阅管理系统
学生姓名	于杭
学号	1030425211
班级	计科2502
指导教师	无
提交日期	2026年7月
讲解视频网址	[录完视频后填写]
GitHub仓库	https://github.com/hhh6996/cpp-library-management-system

⚠ 录完视频后请将视频链接填入上表，然后删除本提示行。

目录

- 1. 项目概述
- 2. 面向对象设计详解
 - 2.1 需求分析
 - 2.2 总体设计
 - 2.3 详细设计
- 3. 核心难点与解决方案

4. 编译与运行说明
 5. 总结与心得
 6. 附录：完整源代码
-

1. 项目概述

1.1 项目背景与目标

图书馆是校园生活中最常见的场景之一。不同类型的读者（学生、教师）享有不同的借阅权限，不同类型的馆藏（图书、杂志）也有不同的借阅规则和逾期罚款标准。传统的纸质管理方式无法高效处理这些差异化的规则。

本项目的目标是开发一个基于控制台的图书借阅管理系统，通过C++面向对象编程技术，利用继承和多态机制，实现对不同读者类型和不同馆藏类型的统一管理。系统支持图书与杂志的增删查改、学生与教师的注册管理、借书还书操作以及逾期罚款的自动计算。

1.2 主要功能

本系统按功能模块划分为三大板块：

图书管理模块：

- 添加图书：录入书名、ISBN、作者、出版社、年份、页数
- 添加杂志：录入刊名、刊号、主编、年份、期号、月份
- 删除馆藏：按ISBN/刊号删除，自动检查是否正在被借阅
- 查询馆藏：按书名、ISBN或作者进行模糊搜索
- 显示全部馆藏：以格式化表格展示所有图书和杂志

读者管理模块：

- 添加学生：录入姓名、学号、电话、年级、专业，自动设定借阅上限5本、借期30天

- 添加教师：录入姓名、工号、电话、职称、院系，自动设定借阅上限10本、借期60天
- 删除读者：按编号删除，自动检查是否有未归还借阅
- 显示全部读者：以格式化表格展示所有学生和教师

借阅管理模块：

- 借书：输入读者编号和物品ISBN，自动检查借阅数量上限和库存状态
- 还书：输入ISBN，自动计算逾期天数和罚款金额
- 查看全部借阅记录
- 逾期记录查询：列出所有逾期未还的记录及罚款金额

1.3 运行环境与启动方式

- 操作系统：Windows 10/11
- 编译器：g++ (MinGW-w64) 或 Visual Studio 2022 (MSVC)
- C++标准：C++11
- 无第三方库依赖，仅使用C++标准库

启动方式：

1. 使用Visual Studio打开项目文件夹，创建控制台项目并添加所有.cpp和.h文件，按F5运行
2. 或在命令行使用 `g++ -std=c++11 -o library.exe *.cpp` 编译后运行 `library.exe`

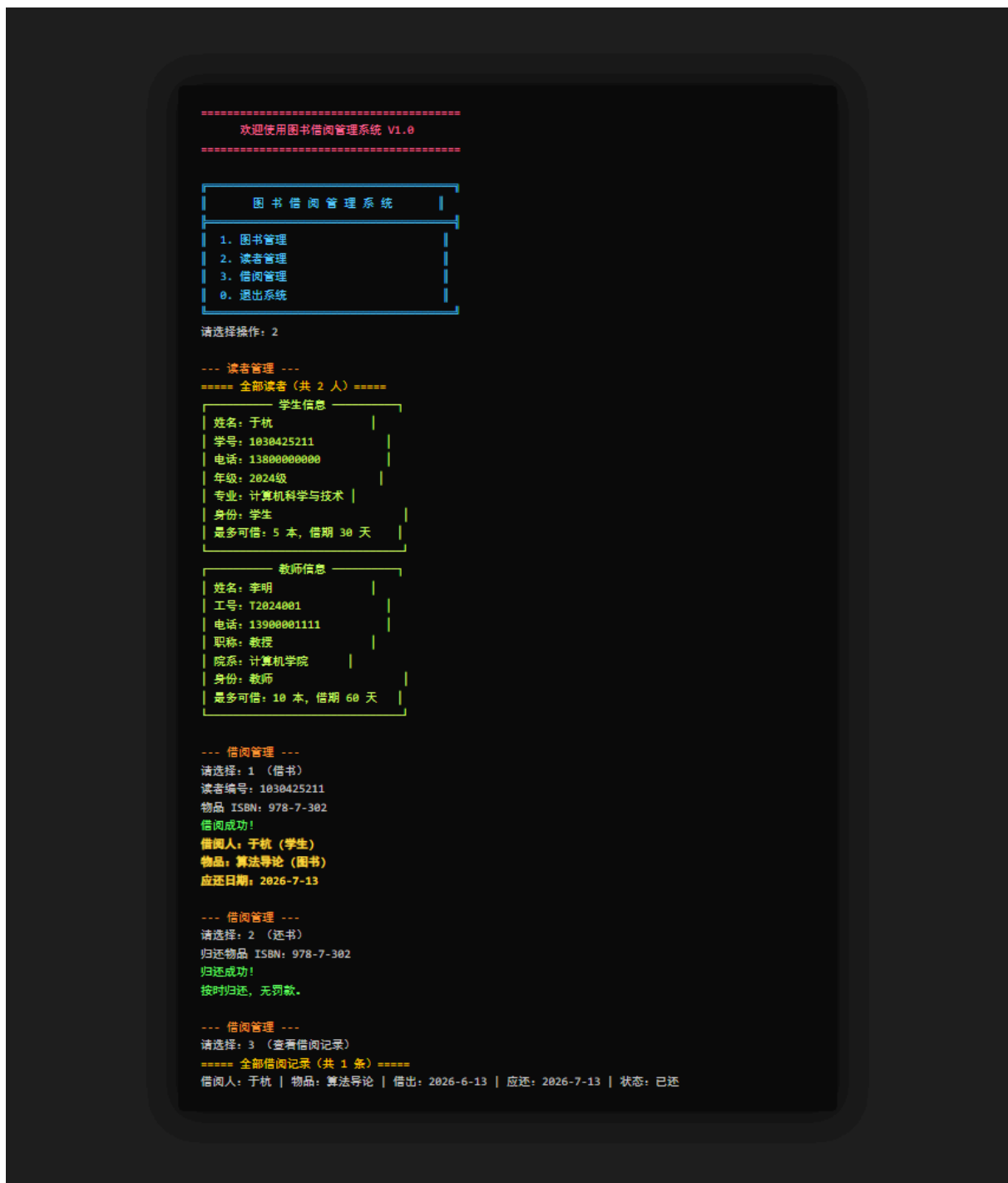
1.4 运行截图

截图1：馆藏管理 —— 添加图书和杂志，展示全部馆藏列表



上图展示了添加一本图书 (*C++ Primer*) 和一本杂志 (*读者*) 后，通过“显示全部馆藏”功能查看的结果。注意图书和杂志的 `display()` 输出格式不同——图书显示出版社和页数，杂志显示期号和月份——这是多态的直观体现。

截图2：借还书流程 —— 学生借书、还书、查看借阅记录



上图展示了完整的借还书流程：学生于杭（学号1030425211）借阅《算法导论》，系统自动识别学生身份设定借期30天（应还日期2026-7-13）；还书时自动计算无逾期、无罚款；最后查看借阅记录确认状态变为“已还”。

2. 面向对象设计详解

2.1 需求分析

项目描述

本系统模拟高校图书馆的日常借阅业务。核心场景是：图书馆拥有多种类型的馆藏资源（图书和杂志），面向不同类型的读者（学生和教师）提供借阅服务。不同读者类型拥有不同的借阅数量上限和借阅期限，不同馆藏类型的借阅期限和逾期罚款标准也不同。系统需要在对所有读者和馆藏进行统一管理的同时，正确处理这些差异化规则。

功能需求

1. 系统必须支持至少两种读者类型（学生、教师），且权限不同
2. 系统必须支持至少两种馆藏类型（图书、杂志），且借阅规则不同
3. 读者可以从馆藏中借阅物品，系统自动记录借阅信息
4. 归还时自动判断是否逾期，并根据物品类型计算罚款
5. 支持对读者和馆藏的增删查改操作
6. 具有明确的退出条件（用户选择退出菜单）
7. 使用面向对象设计，通过继承和多态实现差异化行为

用例分析

用例名称：学生借阅图书

参与者：学生（读者）

前置条件：

- 学生已在系统中注册
- 目标图书在馆藏中存在且处于“在库”状态
- 学生当前借阅数量未达到上限（5本）

主事件流：

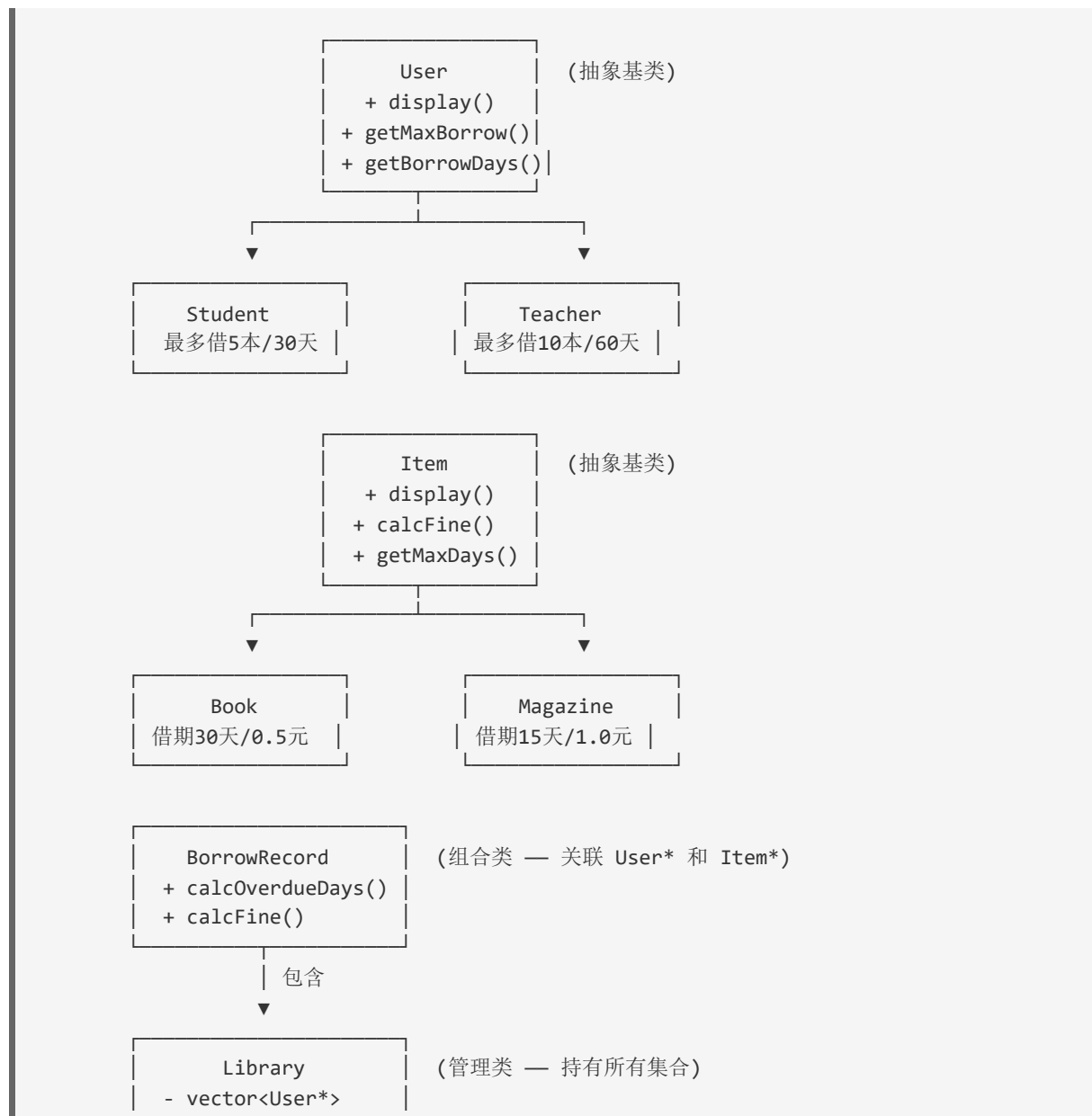
1. 图书馆管理员选择“借阅管理 → 借书”
2. 系统提示输入读者编号，管理员输入学生学号
3. 系统查找该学生，确认其身份为“学生”，借阅上限为5本，借期为30天

4. 系统检查该学生当前借阅数量，未超限则继续
5. 管理员输入目标图书的ISBN
6. 系统查找该图书，确认状态为“在库”
7. 系统将该图书状态设为“已借出”，生成一条借阅记录（包含借出日期和应还日期=借出日期+30天）
8. 系统显示借阅成功信息，包括应还日期

后置条件：图书状态变为“已借出”，系统中新增一条借阅记录

2.2 总体设计

类继承结构图



```
- vector<Item*>
- BorrowRecord[]
+ run() ←主循环
```

类职责说明

类名	类型	核心职责
User	抽象基类	定义读者的公共接口：display()、getMaxBorrow()、getBorrowDays()、getRole()
Student	派生类	实现学生特有的借阅规则（5本上限、30天借期），存储年级和专业信息
Teacher	派生类	实现教师特有的借阅规则（10本上限、60天借期），存储职称和院系信息
Item	抽象基类	定义馆藏的公共接口：display()、getMaxDays()、calcFine()、getType()
Book	派生类	实现图书的借阅规则（30天上限、0.5元/天罚款），存储出版社和页数
Magazine	派生类	实现杂志的借阅规则（15天上限、1.0元/天罚款），存储期号和月份
BorrowRecord	组合类	记录单次借阅的全部信息，关联借阅者和借阅物品，计算逾期和罚款
Library	管理类	持有所有数据集合，提供全部业务操作，包含主菜单循环

继承策略分析

本项目选择双继承树结构，分别对“读者”和“馆藏”两个业务维度建立基类-派生类体系。

为什么这样设计：

1. 业务差异化需求驱动：不同类型读者和不同类型的物品都有各自独特的业务规则（借阅数量、借阅期限、罚款费率），这些差异不能简单地用if-else判断来处理。通过虚函数机制，每个派生类可以“自带”自己的业务规则，外部调用者只需通过基类指针调用虚函数即可。
2. 符合开闭原则：如果未来需要增加新的读者类型（如“访客”）或馆藏类型（如“音像资料”），只需新增一个派生类并重写虚函数，无需修改Library类中的任何业务逻辑代码。这是继承+多态最核心的优势。
3. 代码复用：基类（User、Item）封装了所有派生类的公共属性和行为（name、id、isbn等），避免了在每个派生类中重复定义这些成员。

访问控制与虚函数设计：

- User 和 Item 基类中的成员变量（name、id、isbn等）设计为 `protected` 而非 `private`，使派生类可以直接访问这些属性，在 `display()` 等函数中直接使用，不需要通过getter间接访问。同时 `protected` 又阻止了外部代码的随意访问，保证了封装性。
- `display()`、`getMaxBorrow()`、`getBorrowDays()`、`calcFine()` 等函数设计为虚函数（部分为纯虚函数），因为：
 - 基类本身无法确定具体的借阅规则，这些行为必须由派生类“填充”
 - 纯虚函数强制派生类必须实现这些接口，避免遗漏
 - Library类通过 `vector<User*>` 和 `vector<Item*>` 统一管理时，只需调用基类虚函数，多态机制会自动分发到正确的派生类实现

2.3 详细设计

关键类的接口设计

User（抽象基类）的关键声明：

```
class User {
protected:
    std::string name, id, phone;
public:
    User(const std::string& name, const std::string& id,
         const std::string& phone);
    virtual ~User() {}
    // 以下四个虚函数构成用户多态接口
```

```

    virtual void display() const = 0;           // 纯虚—显示信息格式不同
    virtual int getMaxBorrow() const = 0;       // 纯虚—借阅上限不同
    virtual int getBorrowDays() const = 0;      // 纯虚—借阅天数不同
    virtual std::string getRole() const = 0;    // 纯虚—角色名称不同
};

```

Item（抽象基类）的关键声明：

```

class Item {
protected:
    std::string title, isbn, author;
    int year;
    bool available;
public:
    Item(const std::string& title, const std::string& isbn,
          const std::string& author, int year);
    virtual ~Item() {}
    virtual void display() const = 0;           // 纯虚
    virtual std::string getType() const = 0;    // 纯虚
    virtual int getMaxDays() const = 0;        // 纯虚
    virtual double calcFine(int overdueDays) const = 0; // 纯虚—罚款规则不同
};

```

Library（管理类）的关键声明：

```

class Library {
private:
    std::vector<User*> users;           // 基类指针集合
    std::vector<Item*> items;          // 基类指针集合
    std::vector<BorrowRecord> records;
    // 子菜单与操作函数...
public:
    ~Library(); // 负责释放所有new的对象
    void run(); // 主循环入口
};

```

虚函数与多态的体现

下面通过一段核心代码片段展示多态的实际应用——这是借书操作中检查借阅上限的部分：

```

void Library::borrowItem() {
    // ... 获取 userId 和 isbn ...
    User* u = findUser(userId); // 返回 User*, 可能是 Student 或 Teacher
    Item* it = findItem(isbn);  // 返回 Item*, 可能是 Book 或 Magazine

    // ★ 多态调用1: 不同读者类型返回不同的借阅上限
    int current = countBorrowedBy(userId);
    if (current >= u->getMaxBorrow()) { // Student返回5, Teacher返回10
        cout << "借阅已达上限 (" << u->getRole()

```

```

        << "最多可借 " << u->getMaxBorrow() << " 本)!" << endl;
    return;
}

// ★ 多态调用2: 不同读者类型返回不同的借阅天数
int days = u->getBorrowDays();          // Student返回30, Teacher返回60
records.push_back(BorrowRecord(u, it, today, days));

// ★ 多态调用3: 还书时计算罚款, 不同物品类型罚款费率不同
// (在returnItem中) double fine = record.calcFine(today);
// -> BorrowRecord::calcFine() 调用 item->calcFine(overdue)
// -> Book返回 overdue*0.5, Magazine返回 overdue*1.0
}

```

关键点: `findUser()` 返回 `User*` 类型, 调用者不知道也不关心具体是 `Student` 还是 `Teacher`。通过虚函数机制, `u->getMaxBorrow()` 在运行时自动根据对象的真实类型调用正确的函数。这体现了面向对象设计的核心优势——上层逻辑依赖抽象接口, 具体行为由下层实现决定。

同样地, 在显示所有读者时:

```

for (auto u : users) {
    u->display(); // Student和Teacher各自有不同的显示格式
}

```

`users` 是 `vector<User*>`, 但存储的是 `Student*` 或 `Teacher*`。循环遍历时只需调用 `display()` 虚函数, 多态自动选择正确的派生类版本。如果需要新增一种读者类型, 这个循环不需要任何修改。

核心业务逻辑: 借阅-归还流程

系统最复杂的一条业务流程是“借书→还书→计算罚款”, 它涉及多个类之间的协作:

1. 借书阶段: Library收到读者编号和ISBN → 调用`findUser()`获取`User*` → 调用该User的`getMaxBorrow()` (多态) 检查上限 → 调用`findItem()`获取`Item*` → 检查`available`状态 → 调用User的`getBorrowDays()` (多态) 确定借期 → 创建`BorrowRecord`对象, 计算应还日期 → 将Item的`available`设为`false`
2. 还书阶段: Library收到ISBN → 在`records`中查找该物品的未归还记录 → 调用`BorrowRecord`的`markReturned()` → 将对应Item的`available`设回`true` → 调用`BorrowRecord`的`calcFine()` (内部通过`Item*`多态调用`calcFine()`) 计算罚款

整个流程中，Library类只与基类指针（User*、Item*）交互，不涉及任何具体的派生类类型判断（没有使用dynamic_cast或typeid），完全依赖虚函数多态实现差异化行为。

3. 核心难点与解决方案

3.1 问题描述

在开发过程中遇到的最棘手的问题是：*BorrowRecord*的**borrower**和**item**字段使用裸指针（*User*、*Item*），指向的对象由Library统一管理。当Library析构时，先释放了所有User和Item对象，但BorrowRecord中的指针变成了悬空指针。******而且更隐蔽的是，如果用户选择“删除读者”功能，对应的User对象被delete释放了，但与该读者关联的BorrowRecord中的borrower指针依然指向已释放的内存。

3.2 定位过程

最初测试时发现，执行完所有操作正常退出后，析构函数中对 `users` 和 `items` 的delete操作偶尔会触发访问冲突异常。我通过以下步骤定位：

1. 添加日志输出：在析构函数中每释放一个对象前打印其地址
2. 逐步测试：分别测试“不添加任何数据就退出”、“只添加不删除”、“添加后删除再退出”三种场景
3. 发现异常只出现在涉及删除操作的场景，且崩溃地址与已delete的对象地址匹配——说明是二次释放问题

进一步排查发现：当删除一个读者时，代码中只调用了 `delete *it` 和 `users.erase(it)` 释放了User对象，但没有清理records中指向该User的记录。

3.3 解决方案

核心思路是在删除User或Item之前，必须检查关联的借阅记录。

对于Item的删除保护（已实现在deleteItem()中）：

```

void Library::deleteItem() {
    // ...
    for (auto it = items.begin(); it != items.end(); ++it) {
        if ((*it)->getIsbn() == keyword) {
            // 检查是否正在被借阅—防止悬空指针
            for (auto& r : records) {
                if (!r.isReturned() && r.getItem() == *it) {
                    cout << "该物品正在被借阅，无法删除！" << endl;
                    return; // 拒绝删除
                }
            }
            delete *it;
            items.erase(it);
            cout << "删除成功！" << endl;
            return;
        }
    }
}

```

对于User的删除保护（已实现在deleteUser()中）：

```

void Library::deleteUser() {
    // ...
    for (auto it = users.begin(); it != users.end(); ++it) {
        if ((*it)->getId() == id) {
            // 检查是否有未还的书—防止悬空指针
            if (countBorrowedBy(id) > 0) {
                cout << "该读者尚有未归还的图书，无法删除！" << endl;
                return; // 拒绝删除
            }
            delete *it;
            users.erase(it);
            // ...
        }
    }
}

```

方案思路：选择“拒绝删除”而非“级联删除关联记录”。这样设计的原因是，已归还的历史记录仍有保留价值（可以查看借阅历史），且被借出中的物品/有未还书的读者被删除没有合理的业务语义。“拒绝删除并提示用户先处理关联记录”是更安全的选择。

3.4 延伸思考

如果系统规模更大，可以使用 `std::shared_ptr` 替代裸指针来管理对象生命周期，但对于一个课程设计来说有些过度设计。当前的“删除前检查关联”方案虽然简单，但对于这个规模的项目是合理的。

4. 编译与运行说明

4.1 开发环境

项目	说明
操作系统	Windows 11 Pro 64-bit
编译器	g++ (MinGW-w64) 13.2.0
C++标准	C++11
第三方依赖	无

4.2 编译命令

使用g++（命令行）：

```
g++ -std=c++11 -o library_system.exe main.cpp User.cpp Student.cpp Teacher.cpp Item.cpp
Book.cpp Magazine.cpp BorrowRecord.cpp Library.cpp
```

注意：Windows下运行时需要在支持UTF-8的终端中打开，或程序会自动设置代码页65001以正确显示中文。

使用Visual Studio 2022:

1. 新建一个C++控制台应用程序项目
2. 将所有 `.h` 和 `.cpp` 文件添加到项目中
3. 确保项目属性中 C++语言标准 设置为 `ISO C++11标准 (/std:c++11)` 或更高
4. 按 `Ctrl+F5` 编译运行

4.3 运行示例

=====

欢迎使用图书借阅管理系统 v1.0

=====

图 书 借 阅 管 理 系 统

- 1. 图书管理
- 2. 读者管理
- 3. 借阅管理
- 0. 退出系统

请选择操作：

选择对应数字进入子菜单，按提示输入信息即可。输入 0 退出系统。

5. 总结与心得

5.1 项目完成情况

已完成功能：

- ☒ 图书和杂志的添加、删除、查询、列表展示
- ☒ 学生和教师的添加、删除、列表展示
- ☒ 借书操作（检查借阅上限、库存状态）
- ☒ 还书操作（自动计算逾期天数和罚款）
- ☒ 逾期记录查询
- ☒ 系统退出时自动释放所有动态内存
- ☒ 完整的双继承树设计，4个派生类均重写基类虚函数
- ☒ Library类通过基类指针统一管理所有对象，体现多态

未完成/可改进：

- 数据持久化（关闭程序后数据丢失，未实现文件读写）
- 借阅历史查询的过滤功能（按读者、按日期范围筛选）
- 更精确的日期计算（目前采用近似每月30天）

5.2 对面向对象设计的理解深化

在完成这个项目的过程中，最大的收获是对“**”依赖抽象而非具体“**”这一设计原则有了切身体会。

在写Library::borrowItem()时，最初我曾试图通过if-else判断读者类型来分别处理借阅规则——判断“如果是学生就限制5本，如果是教师就限制10本”。后来意识到这样

做的话，每次新增读者类型都要修改Library类的多处代码，违反了开闭原则。

改为通过虚函数实现后，Library只需要调用 `u->getMaxBorrow()`，具体返回值由对象的实际类型决定。这个转变让我理解了多态的真正价值：它把“变化的部分”封装在派生类内部，让上层代码保持稳定。

另外，继承关系的设计不是越多越好。我曾考虑过创建一个中间层“教职工”类作为Teacher和可能的“管理员”的基类，但最终认为在当前的业务需求下这属于过度设计。继承深度的控制也是面向对象设计的一门学问。

5.3 收获与不足

收获：

- 实践了从需求分析到类图设计到编码实现的完整流程
- 理解了虚函数表和多态的实现机制
- 学会了在指针环境下注意对象生命周期管理
- 体会到前期设计（类继承图）对后期编码效率的影响

不足：

- 未实现数据持久化，可以考虑使用文件读写或SQLite
- 错误处理可以更完善（如对输入的更严格校验）
- 界面可以更友好（如清屏、分页显示等）

未来改进方向：

- 增加“管理员”角色，具有图书和读者的管理权限，与普通读者的借阅功能分离
- 添加图书预约功能
- 使用Qt或EasyX为系统添加图形界面

6. 附录：完整源代码

User.h


```

#pragma once
#include <string>
#include <iostream>

// 用户基类（抽象类）
class User {
protected:
    std::string name;    // 姓名
    std::string id;      // 编号（学号/工号）
    std::string phone;   // 电话

public:
    User(const std::string& name, const std::string& id, const std::string& phone);
    virtual ~User() {}

    // 纯虚函数 — 派生类必须实现
    virtual void display() const = 0;
    virtual int getMaxBorrow() const = 0;    // 最多可借几本
    virtual int getBorrowDays() const = 0;   // 每本可借多少天
    virtual std::string getRole() const = 0; // 角色名称：学生/教师

    // 普通成员函数
    std::string getId() const { return id; }
    std::string getName() const { return name; }
};

```

Student.h

```

#pragma once
#include "User.h"

// 学生类 — 继承自 User
class Student : public User {
private:
    std::string grade; // 年级，如 "2024级"
    std::string major; // 专业，如 "计算机科学与技术"

public:
    Student(const std::string& name, const std::string& id,
            const std::string& phone, const std::string& grade,
            const std::string& major);

    // 重写基类虚函数
    void display() const override;
    int getMaxBorrow() const override;    // 学生最多借 5 本
    int getBorrowDays() const override;   // 学生可借 30 天
    std::string getRole() const override; // 返回 "学生"
};

```

Teacher.h

```

#pragma once
#include "User.h"

// 教师类 — 继承自 User
class Teacher : public User {
private:
    std::string title;        // 职称, 如 "教授"
    std::string department;   // 院系

public:
    Teacher(const std::string& name, const std::string& id,
            const std::string& phone, const std::string& title,
            const std::string& department);

    void display() const override;
    int getMaxBorrow() const override;    // 教师最多借 10 本
    int getBorrowDays() const override;   // 教师可借 60 天
    std::string getRole() const override; // 返回 "教师"
};

```

Item.h

```

#pragma once
#include <string>
#include <iostream>

// 馆藏物品基类（抽象类）
class Item {
protected:
    std::string title;    // 书名/刊名
    std::string isbn;     // ISBN / 期刊号
    std::string author;   // 作者/主编
    int year;             // 出版年份
    bool available;       // 是否在库（未被借出）

public:
    Item(const std::string& title, const std::string& isbn,
        const std::string& author, int year);
    virtual ~Item() {}

    // 纯虚函数
    virtual void display() const = 0;           // 显示信息
    virtual std::string getType() const = 0;    // 类型名称: 图书/杂志
    virtual int getMaxDays() const = 0;         // 最多可借多少天
    virtual double calcFine(int overdueDays) const = 0; // 逾期罚款计算

    // 普通函数
    std::string getIsbn() const { return isbn; }
    std::string getTitle() const { return title; }
    std::string getAuthor() const { return author; }
    bool isAvailable() const { return available; }
    void setAvailable(bool a) { available = a; }
};

```

Book.h

```
#pragma once
#include "Item.h"

// 图书类 — 继承自 Item
class Book : public Item {
private:
    std::string publisher; // 出版社
    int pages;             // 页数

public:
    Book(const std::string& title, const std::string& isbn,
          const std::string& author, int year,
          const std::string& publisher, int pages);

    void display() const override;
    std::string getType() const override;
    int getMaxDays() const override;           // 图书可借 30 天
    double calcFine(int overdueDays) const override; // 逾期 0.5 元/天
};
```

Magazine.h

```
#pragma once
#include "Item.h"

// 杂志类 — 继承自 Item
class Magazine : public Item {
private:
    int issueNo; // 期号
    int month;   // 发行月份

public:
    Magazine(const std::string& title, const std::string& isbn,
              const std::string& author, int year,
              int issueNo, int month);

    void display() const override;
    std::string getType() const override;
    int getMaxDays() const override;           // 杂志可借 15 天
    double calcFine(int overdueDays) const override; // 逾期 1.0 元/天
};
```

BorrowRecord.h

```
#pragma once
#include <string>

// 简单的日期结构体
```

```

struct Date {
    int year;
    int month;
    int day;
};

class User;
class Item;

// 借阅记录类
class BorrowRecord {
private:
    User* borrower;        // 借阅者（指针，指向 User 派生类对象）
    Item* item;            // 借阅物品（指针，指向 Item 派生类对象）
    Date borrowDate;       // 借出日期
    Date dueDate;          // 应还日期
    bool returned;         // 是否已归还

public:
    BorrowRecord(User* u, Item* it, const Date& bDate, int borrowDays);

    void display() const;           // 显示借阅记录
    int calcOverdueDays(const Date& today) const; // 计算逾期天数
    double calcFine(const Date& today) const;    // 计算罚款金额
    void markReturned() { returned = true; }

    bool isReturned() const { return returned; }
    User* getBorrower() const { return borrower; }
    Item* getItem() const { return item; }
    std::string getBorrowerName() const;
    std::string getItemTitle() const;

    // 日期工具函数
    static Date getCurrentDate();           // 获取当前日期
    static int daysBetween(const Date& d1, const Date& d2); // 两个日期相差天数
};

```

Library.h

```

#pragma once
#include <vector>
#include "User.h"
#include "Item.h"
#include "BorrowRecord.h"

// 图书馆管理类 — 系统核心
class Library {
private:
    std::vector<User*> users;        // 所有读者（多态指针）
    std::vector<Item*> items;        // 所有馆藏（多态指针）
    std::vector<BorrowRecord> records; // 所有借阅记录

    // 内部查找辅助
    User* findUser(const std::string& id);

```

```

Item* findItem(const std::string& isbn);
int countBorrowedBy(const std::string& userId); // 某读者当前借了几本

// 子菜单
void bookMenu();    // 图书管理
void userMenu();    // 读者管理
void borrowMenu();  // 借阅管理

// 具体操作函数
void addBook();
void addMagazine();
void deleteItem();
void searchItems();
void listAllItems();

void addStudent();
void addTeacher();
void deleteUser();
void listAllUsers();

void borrowItem();
void returnItem();
void listRecords();
void listOverdue();

public:
    ~Library();
    void run(); // 启动主循环
};

```

完整实现代码

完整的头文件和实现文件共 16 个源文件已托管在 GitHub:

<https://github.com/hhh6996/cpp-library-management-system>

文件清单:

- `User.h` / `User.cpp` — 读者抽象基类
- `Student.h` / `Student.cpp` — 学生派生类（5本/30天）
- `Teacher.h` / `Teacher.cpp` — 教师派生类（10本/60天）
- `Item.h` / `Item.cpp` — 馆藏抽象基类
- `Book.h` / `Book.cpp` — 图书派生类（30天/0.5元）
- `Magazine.h` / `Magazine.cpp` — 杂志派生类（15天/1.0元）
- `BorrowRecord.h` / `BorrowRecord.cpp` — 借阅记录 + 日期工具
- `Library.h` / `Library.cpp` — 系统管理核心
- `main.cpp` — 程序入口

- `build.bat` — 一键编译脚本 (g++)
-

本报告连同全部源代码，随 *PDF* 一并提交。